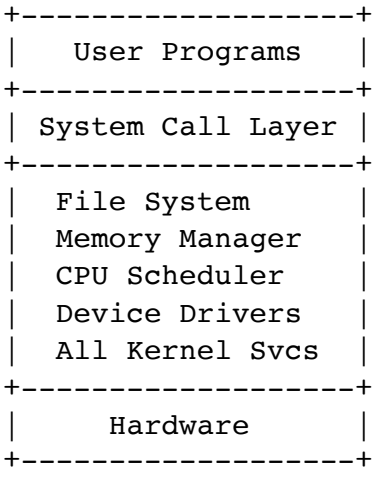
Q1

1. Draw and explain the Monolithic, Microkernel, and Layered operating system architectures.

(a) Monolithic Architecture

Diagram



Explanation

A monolithic operating system places all major OS services inside a single kernel space. All components such as memory management, file systems, process scheduling, and device drivers execute together in kernel mode.

Advantages

- High performance due to direct communication between services.
- Faster execution because there is minimal overhead.
- Easier interaction among kernel components.

Disadvantages

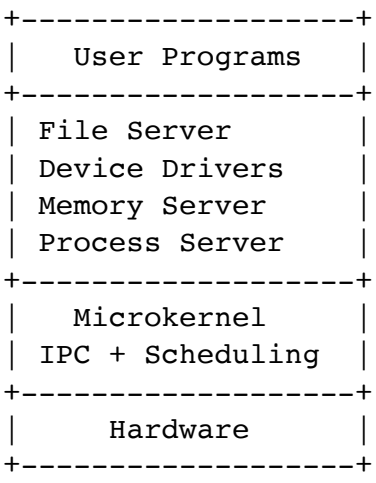
- Difficult to maintain and debug.
- Failure in one component may crash the whole system.
- Large kernel size reduces reliability.

Examples

- UNIX
- Linux

(b) Microkernel Architecture

Diagram



Explanation

In a microkernel architecture, only essential services such as communication, scheduling, and memory protection remain inside the kernel. Other services run in user space.

Advantages

- Better reliability and security.
- Easier to extend and maintain.
- Fault isolation is improved.

Disadvantages

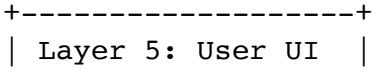
- Communication overhead due to message passing.
- Lower performance compared to monolithic systems.

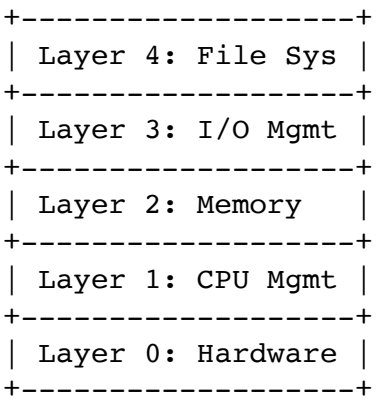
Examples

- MINIX
- QNX
- Mach

(c) Layered Architecture

Diagram





Explanation

In the layered approach, the operating system is divided into different layers. Each layer uses the services of the layer below it and provides services to the layer above.

Advantages

- Easier debugging and testing.
- Better modularity.
- Simpler maintenance.

Disadvantages

- Layer communication overhead.
- Difficult to define proper layers.
- Performance may decrease.

Examples

- THE Operating System
- Some modern UNIX variants partially follow layered design.

2. Why is a Real-Time Operating System (RTOS) preferred in air traffic control systems?

A Real-Time Operating System is preferred in air traffic control systems because such systems require deterministic and time-critical responses.

Reasons

1. **Guaranteed Response Time**
   Air traffic control systems must respond within strict deadlines. RTOS ensures predictable timing.

2. **High Reliability**
   Failures can lead to catastrophic accidents. RTOS systems are designed for reliability.
3. **Priority-Based Scheduling**
   Critical tasks such as aircraft tracking and collision alerts receive immediate CPU access.
4. **Minimal Latency**
   RTOS minimizes interrupt and scheduling delays.
5. **Continuous Operation**
   Air traffic control requires uninterrupted 24×7 operation.

Examples of RTOS

- VxWorks
- QNX
- RTLinux

Q2

Why are system calls necessary in an operating system?

System calls provide an interface between user programs and the operating system kernel. User programs cannot directly access hardware resources because direct access would compromise security and system stability.

Necessity of System Calls

1. Controlled access to hardware.
2. Protection and security.
3. Resource management.
4. Process and memory control.
5. File and device operations.

Steps involved when a user program performs a system call

Step-by-Step Execution

```
User Program
     |
System Call Invocation
     |
```

```
Trap/Interrupt to Kernel Mode
      |
Kernel Executes Requested Service
      |
Result Returned to User Program
```

Detailed Explanation

1. The user program calls a library function such as `read()`.
2. Parameters are placed in registers or stack.
3. A software interrupt (trap) transfers control to the kernel.
4. CPU mode changes from user mode to kernel mode.
5. Kernel identifies the requested service.
6. Kernel executes the service.
7. Result is returned.
8. Control goes back to user mode.

Illustration Example

```
fd = open("data.txt", O_RDONLY);
read(fd, buffer, 100);
close(fd);
```

Here, `open`, `read`, and `close` are system calls.

Q3

Explain each system call involved to implement:

```
copy file1.txt file2.txt
```

This command copies contents from `file1.txt` to `file2.txt`.

Sequence of System Calls

Step 1: Open Source File

```
fd1 = open("file1.txt", O_RDONLY);
```

Purpose

Opens the source file in read-only mode.

Step 2: Create/Open Destination File

```
fd2 = open("file2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Purpose

- Opens destination file for writing.
- Creates file if it does not exist.
- Clears existing content.

Step 3: Read Data

```
n = read(fd1, buffer, SIZE);
```

Purpose

Reads data from source file into buffer.

Step 4: Write Data

```
write(fd2, buffer, n);
```

Purpose

Writes buffer data into destination file.

Step 5: Repeat Until EOF

The read and write operations continue until `read()` returns 0.

Step 6: Close Files

```
close(fd1);
close(fd2);
```

Purpose

Releases file descriptors and resources.

Complete Flow

```
open(source)
open(destination)
while(read)
    write
close(source)
close(destination)
```

Q4

1. Taking suitable example, explain Instruction Interleaving.

Instruction interleaving refers to the execution of instructions from multiple processes in an overlapping manner by the CPU.

Example

Assume two processes:

```
P1: A1 A2 A3
P2: B1 B2 B3
```

Possible interleaving:

```
A1 B1 A2 B2 A3 B3
```

Explanation

The operating system rapidly switches between processes using context switching, creating the illusion of simultaneous execution.

Importance

- Improves CPU utilization.
- Supports multiprogramming.
- Enables concurrency.

2. Prove SJF yields the lowest overall waiting time.

Shortest Job First (SJF)

SJF selects the process with the smallest CPU burst time first.

Example

Process	Burst Time
P1	6
P2	2
P3	8
P4	3

FCFS Scheduling

Execution Order:

```
P1 → P2 → P3 → P4
```

Waiting Times:

Process	Waiting Time
P1	0
P2	6
P3	8
P4	16

Average Waiting Time:

```
(0 + 6 + 8 + 16) / 4 = 7.5
```

SJF Scheduling

Execution Order:

```
P2 → P4 → P1 → P3
```

Waiting Times:

Process	Waiting Time
P2	0
P4	2
P1	5
P3	11

Average Waiting Time:

```
(0 + 2 + 5 + 11) / 4 = 4.5
```

Conclusion

SJF minimizes average waiting time because shorter processes complete earlier, reducing total waiting experienced by all processes.

Q5

1. Differentiate between Program and Process

Program	Process
Passive entity	Active entity
Stored on disk	Exists in memory
Static	Dynamic
Contains instructions	Program under execution
No execution state	Has state, PCB, registers

2. Differentiate between Interrupt and System Call

Interrupt	System Call
Generated by hardware/software events	Generated by user programs
Usually asynchronous	Synchronous
Used for urgent attention	Used to request OS services
Example: keyboard interrupt	Example: read(), write()

3. Differentiate between argc and argv

argc	argv
Argument count	Argument vector
Integer variable	Array of strings
Stores number of command-line arguments	Stores actual arguments

Example

```
./prog hello world
```

```
argc = 3
argv[0] = ./prog
```

```
argv[1] = hello
argv[2] = world
```

4. Differentiate between Dispatcher and Scheduler

Dispatcher	Scheduler
Gives CPU to selected process	Selects next process
Performs context switching	Maintains scheduling policy
Faster component	Decision-making component
Works after scheduling	Works before dispatching

Q6

A system uses preemptive priority scheduling where larger priority numbers indicate higher priority.

- New processes enter with priority 0.
- While waiting in ready queue, priority changes at rate `a`.
- While running on CPU, priority changes at rate `b`.

1. What scheduling algorithm results from b > a > 0?

Answer

This results in **First Come First Served (FCFS)** behavior.

Justification

- The running process gains priority faster than waiting processes.
- Therefore, once a process starts execution, its priority continues increasing rapidly.
- Waiting processes cannot overtake the running process.
- Hence, the currently running process continues until completion.

Illustration

```
Running process priority increase = b
Waiting process priority increase = a
Where b > a
```

So the running process always remains ahead.

2. What scheduling algorithm results from a > b > 0?

Answer

This results in **Round Robin-like / Aging-based preemptive scheduling**.

Justification

- Waiting processes gain priority faster than the running process.
- Eventually, waiting processes overtake the currently running process.
- The CPU is preempted repeatedly.
- This behavior resembles Round Robin with dynamic priorities.

Illustration

```
Waiting priority increase = a
Running priority increase = b
Where a > b
```

Thus, long waiting processes eventually get CPU time.

Q7

Given

Three processes arrive at time 0.

Process	Total Time
P1	10
P2	20
P3	30

Each process spends:

- First 20% in I/O
- Next 70% in CPU
- Last 10% in I/O

Scheduling algorithm: Shortest Remaining Compute Time First.

Step 1: Calculate CPU and I/O Times

P1

Total = 10

- Initial I/O = 20% of 10 = 2
- CPU = 70% of 10 = 7
- Final I/O = 10% of 10 = 1

P2

Total = 20

- Initial I/O = 4
- CPU = 14
- Final I/O = 2

P3

Total = 30

- Initial I/O = 6
- CPU = 21
- Final I/O = 3

Step 2: Timeline Analysis

Time 0–2

All processes are performing initial I/O.

CPU is idle from 0 to 2.

Time 2–9

P1 completes I/O first and executes CPU burst.

CPU busy for 7 units.

Time 9–23

P2 executes CPU burst.

CPU busy for 14 units.

Time 23–44

P3 executes CPU burst.

CPU busy for 21 units.

Step 3: Total Time

Total elapsed time:

```
44 + final I/O of P3
= 44 + 3
= 47
```

CPU idle time:

```
2 units
```

Step 4: Percentage CPU Idle

```
CPU Idle Percentage
= (Idle Time / Total Time) × 100
= (2 / 47) × 100
≈ 4.26%
```

Final Answer

```
CPU remains idle for approximately 4.26% of the total time.
```